

MODULE 2 – PRACTICE EXERCISES

SQL FUNDAMENTALS (DDL & DML)

Name	Aavihotr Anand Jha
Employee ID	90119991

Level 1

Q1. (Core)

1. PAN number — CHAR(10): the value is always exactly 10 characters.
2. Interest amount — NUMERIC(15,2): monetary calculations need exact decimal arithmetic.
3. Deposit days — SMALLINT: the maximum of 3,650 fits easily in a small integer.
4. UPI authorization moment — TIMESTAMPTZ: records the exact date/time with time-zone information.
5. Loan instalment due date — DATE: only the calendar date is required.
6. Branch complaint note — TEXT: suitable for variable-length free text.
7. Lifetime transaction counter — BIGINT: allows a very large number of transactions.
8. Auto-renew — BOOLEAN: represents a yes/no value.

Q2. (Core)

FLOAT can represent decimal values approximately, while NUMERIC is designed for exact decimal arithmetic. For banking money, exact arithmetic avoids small rounding errors that can affect balances, interest and reconciliation.

SQL:

```
SELECT 0.1::FLOAT8 + 0.2::FLOAT8 AS float_result;  
SELECT 0.1::NUMERIC + 0.2::NUMERIC AS numeric_result;
```

Output :

Typical PostgreSQL output is 0.30000000000000004 for FLOAT8 and 0.3 for NUMERIC.

Q3.

Constraint	What it guarantees	ICICI Bank example
NOT NULL	A value must be supplied	customer.name
UNIQUE	Duplicate values are prevented	account.account_number
PRIMARY KEY	Uniquely identifies each row and cannot be NULL	customer.customer_id
FOREIGN KEY	Referenced parent record must exist	account.customer_id → customer.customer_id
CHECK	A value/row must satisfy a condition	monthly_amount >= 500
DEFAULT	Supplies a value when one is omitted	start_date DEFAULT CURRENT_DATE

Q4. (Core)

A PRIMARY KEY cannot be NULL, uniquely identifies each row, and a table has one primary-key constraint. A UNIQUE constraint also enforces uniqueness, but a table can have multiple UNIQUE constraints and PostgreSQL permits multiple NULLs under a normal UNIQUE constraint. deposit_id is an appropriate primary key because it is a stable internal identifier for the deposit record; receipt_number can remain a unique business-facing identifier.

Q5. (Core)

DELETE removes rows, supports WHERE, can be rolled back inside a transaction, fires DELETE triggers, and is generally slower when removing a large number of rows.

TRUNCATE removes all rows from a table, has no WHERE clause, can be rolled back in PostgreSQL when transactional, does not fire ordinary row-level DELETE triggers, and is generally much faster for clearing a table.

DROP removes the table object itself, has no WHERE clause, can be transactional in PostgreSQL, removes the table definition and associated data/dependencies according to the dependency rules, and is generally fast because the object is removed rather than individual rows.

Q6.

The job violates the seven-year retention requirement because it deletes records older than only 90 days. It can also destroy data required for audit, investigation, reconciliation and regulatory reporting. The bank should retain at least seven years of authoritative transaction history and use a controlled archive/partitioning strategy with documented retention and recovery procedures.

Q7. (Core)

```
SELECT NULL = NULL;      → NULL
SELECT NULL <> NULL;     → NULL
SELECT 5000 + NULL;      → NULL
SELECT COALESCE(NULL, NULL, 'FALLBACK'); → FALLBACK
SELECT 'A' || NULL || 'B'; → NULL
```

NULL represents an unknown/missing value, so comparisons and arithmetic involving NULL normally produce NULL. COALESCE returns the first non-NULL expression.

Q8.

AND has higher precedence than OR. Thus:

`branch_id = 1 OR branch_id = 2 AND principal_amount > 500000`

means `branch_id = 1 OR (branch_id = 2 AND principal_amount > 500000)`.

The parenthesized version means `(branch_id = 1 OR branch_id = 2) AND principal_amount > 500000`.

The first query therefore includes all branch-1 rows, while the second requires the amount condition for both branches.

Q9. (Core)

If `txn_date` is a `TIMESTAMP`, `BETWEEN DATE '2026-01-01' AND DATE '2026-03-31'` includes only midnight at the beginning of 31 March, so transactions later on 31 March are missed. Use a half-open interval:

```
WHERE txn_date >= TIMESTAMP '2026-01-01 00:00:00'
```

```
AND txn_date <  TIMESTAMP '2026-04-01 00:00:00'
```

If `txn_date` is actually a `DATE` column, the original `BETWEEN` predicate is inclusive and is not defective for the stated dates.

Q10. (Core)

WHERE is evaluated before the SELECT list is produced, so a SELECT-list alias normally does not exist yet when WHERE is evaluated. ORDER BY is evaluated after the SELECT expressions, so the alias is available there.

Level 2

Q11. (Core)

SQL:

```
CREATE TABLE recurring_deposit (  
    rd_id INTEGER CONSTRAINT pk_recurring_deposit PRIMARY KEY,  
    customer_id INTEGER NOT NULL CONSTRAINT fk_rd_customer  
        REFERENCES customer(customer_id),  
    branch_id INTEGER NOT NULL CONSTRAINT fk_rd_branch  
        REFERENCES branch(branch_id),  
    rd_number VARCHAR(20) NOT NULL CONSTRAINT uq_rd_number UNIQUE,  
    monthly_amount NUMERIC(15,2) NOT NULL  
        CONSTRAINT ck_rd_monthly_amount CHECK (monthly_amount >= 500),  
    interest_rate NUMERIC(5,2) NOT NULL  
        CONSTRAINT ck_rd_interest_rate CHECK (interest_rate > 0 AND interest_rate <= 12),  
    tenure_months INTEGER NOT NULL  
        CONSTRAINT ck_rd_tenure CHECK (tenure_months BETWEEN 6 AND 120),  
    start_date DATE NOT NULL DEFAULT CURRENT_DATE,  
    maturity_date DATE NOT NULL,  
    installments_paid INTEGER NOT NULL DEFAULT 0  
        CONSTRAINT ck_rd_installments_paid CHECK (installments_paid >= 0),  
    status VARCHAR(10) NOT NULL DEFAULT 'ACTIVE'  
        CONSTRAINT ck_rd_status CHECK (status IN  
            ('ACTIVE', 'MATURED', 'DEFAULTED', 'CLOSED')),  
    CONSTRAINT ck_rd_maturity_after_start CHECK (maturity_date > start_date)  
);
```

Output:

PostgreSQL does not treat DEFAULT as a separately named constraint, so the defaults are declared inline. All named integrity constraints are explicitly named.

Q12. (Core)

SQL:

```
-- 1. monthly_amount below minimum  
INSERT INTO recurring_deposit  
(rd_id,customer_id,branch_id,rd_number,monthly_amount,interest_rate,tenure_months,start_date,maturity_date)  
VALUES (9001,1001,1,'BADRD9001',100,7,12,DATE '2026-03-01',DATE '2027-03-01');  
  
-- 2. non-existent customer  
INSERT INTO recurring_deposit  
(rd_id,customer_id,branch_id,rd_number,monthly_amount,interest_rate,tenure_months,start_date,maturity_date)  
VALUES (9002,5555,1,'BADRD9002',1000,7,12,DATE '2026-03-01',DATE '2027-03-01');  
  
-- 3. maturity before start  
INSERT INTO recurring_deposit  
(rd_id,customer_id,branch_id,rd_number,monthly_amount,interest_rate,tenure_months,start_date,maturity_date)  
VALUES (9003,1001,1,'BADRD9003',1000,7,12,DATE '2026-03-10',DATE '2026-03-01');  
  
-- 4. duplicate rd_number (run once successfully, then repeat with another rd_id)  
INSERT INTO recurring_deposit  
(rd_id,customer_id,branch_id,rd_number,monthly_amount,interest_rate,tenure_months,start_date,maturity_date)  
VALUES (9004,1001,1,'BADRD9004',1000,7,12,DATE '2026-03-01',DATE '2027-03-01');  
  
INSERT INTO recurring_deposit  
(rd_id,customer_id,branch_id,rd_number,monthly_amount,interest_rate,tenure_months,start_date,maturity_date)
```

```
VALUES (9005,1005,5,'BADRD9004',1000,7,12,DATE '2026-03-01',DATE '2027-03-01');

-- 5. invalid status
INSERT INTO recurring_deposit
(rd_id,customer_id,branch_id,rd_number,monthly_amount,interest_rate,tenure_months,start_date,maturity_date,status)
VALUES (9006,1001,1,'BADRD9006',1000,7,12,DATE '2026-03-01',DATE '2027-03-01','PAUSED');

-- 6. negative installments
INSERT INTO recurring_deposit
(rd_id,customer_id,branch_id,rd_number,monthly_amount,interest_rate,tenure_months,start_date,maturity_date,installments_paid)
VALUES (9007,1001,1,'BADRD9007',1000,7,12,DATE '2026-03-01',DATE '2027-03-01',-3);
```

Output :

All six are intended to fail. Record the exact PostgreSQL ERROR text from your own database. Use a transaction/rollback around the test data.

Q13.

SQL:

```
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate,
tenure_months, start_date, maturity_date)
VALUES
(8001,1001,1,'RD2026000001',5000,7.10,24,DATE '2026-03-01',DATE '2028-03-01'),
(8002,1005,5,'RD2026000002',2500,6.85,12,DATE '2026-03-05',DATE '2027-03-05'),
(8003,1012,4,'RD2026000003',10000,7.35,60,DATE '2026-03-10',DATE '2031-03-10');
```

Q14.

SQL:

```
ALTER TABLE recurring_deposit
ADD COLUMN nominee_name VARCHAR(60);

UPDATE recurring_deposit
SET nominee_name = 'Meera Reddy'
WHERE rd_id = 8001;

SELECT rd_id, nominee_name
FROM recurring_deposit
WHERE rd_id = 8001;
```

Q15.

SQL:

```
SELECT *
FROM recurring_deposit
WHERE monthly_amount < 1000;

ALTER TABLE recurring_deposit
DROP CONSTRAINT ck_rd_monthly_amount;

ALTER TABLE recurring_deposit
ADD CONSTRAINT ck_rd_monthly_amount
CHECK (monthly_amount >= 1000);
```

Output:

Run the SELECT first. If it returns rows, the new rule would make those rows invalid; resolve the data before replacing the constraint.

Q16.

SQL:

```
-- Required failed attempt on the populated table:
ALTER TABLE recurring_deposit
ADD COLUMN scheme_code VARCHAR(8) NOT NULL;

-- Correct safe approach:
ALTER TABLE recurring_deposit
ADD COLUMN scheme_code VARCHAR(8);

UPDATE recurring_deposit
SET scheme_code = 'RDSTD'
WHERE scheme_code IS NULL;

ALTER TABLE recurring_deposit
ALTER COLUMN scheme_code SET NOT NULL;

SELECT COUNT(*) AS null_scheme_codes
FROM recurring_deposit
WHERE scheme_code IS NULL;
```

Output:

The first ALTER TABLE should fail because existing rows would receive NULL for a new NOT NULL column. Add it nullable, populate every existing row with a valid code, then make it NOT NULL.

Q17. (Core)

SQL:

```
SELECT receipt_number, principal_amount AS principal, rate, maturity_date
FROM fixed_deposit
WHERE principal_amount >= 500000
ORDER BY principal_amount DESC;
```

Q18.

SQL:

```
SELECT *
FROM fixed_deposit
WHERE booked_date >= DATE '2026-03-01'
AND booked_date < DATE '2026-03-11'
ORDER BY booked_date, principal_amount DESC;
```

Q19. (Core)

SQL:

```
SELECT *
FROM fixed_deposit
WHERE nominee_name IS NULL;

SELECT *
FROM fixed_deposit
WHERE nominee_name IS NULL
OR nominee_name <> 'Priya Verma';
```

Output :

The second query needs more than <> because comparisons with NULL produce UNKNOWN, and WHERE retains only TRUE conditions.

Q20. (Core)

SQL:

```
SELECT receipt_number,
       principal_amount AS principal,
       ROUND((principal_amount * rate / 100)::numeric, 2) AS annual_interest
FROM fixed_deposit
ORDER BY annual_interest DESC
LIMIT 5;
```

Q21.

SQL:

```
SELECT receipt_number,
       customer_name,
       principal_amount AS principal,
       CASE
         WHEN principal_amount >= 1000000 THEN 'HIGH VALUE'
         WHEN principal_amount >= 100000 THEN 'STANDARD'
         ELSE 'SMALL'
       END AS segment
FROM fixed_deposit
ORDER BY
  CASE
    WHEN principal_amount >= 1000000 THEN 1
    WHEN principal_amount >= 100000 THEN 2
    ELSE 3
  END,
  principal_amount DESC;
```

Output:

If the supplied schema does not have customer_name directly in fixed_deposit, join fixed_deposit to customer using the actual customer_id column and select the customer's name.

Q22.

SQL:

```
-- PostgreSQL
SELECT txn_id, account_id, amount, narration
FROM account_txn
WHERE narration ILIKE '%UPI%' AND amount > 50000
ORDER BY amount DESC
LIMIT 10;

-- SQL Server
SELECT TOP (10) txn_id, account_id, amount, narration
FROM account_txn
WHERE narration LIKE '%UPI%' AND amount > 50000
ORDER BY amount DESC;

-- Oracle
SELECT txn_id, account_id, amount, narration
FROM account_txn
WHERE UPPER(narration) LIKE '%UPI%' AND amount > 50000
ORDER BY amount DESC
FETCH FIRST 10 ROWS ONLY;
```

Q23.

SQL:

```

BEGIN;

SELECT *
FROM fixed_deposit
WHERE deposit_id = 7009;

UPDATE fixed_deposit
SET status = 'BROKEN',
    auto_renew = 'N'
WHERE deposit_id = 7009;

SELECT deposit_id, status, auto_renew
FROM fixed_deposit
WHERE deposit_id = 7009;

ROLLBACK;

```

Q24. (Core)

SQL:

```

SELECT *
FROM fixed_deposit
WHERE status = 'ACTIVE'
    AND maturity_date <= DATE '2026-06-30';

SELECT COUNT(*) AS to_be_matured
FROM fixed_deposit
WHERE status = 'ACTIVE'
    AND maturity_date <= DATE '2026-06-30';

BEGIN;

UPDATE fixed_deposit
SET status = 'MATURED'
WHERE status = 'ACTIVE'
    AND maturity_date <= DATE '2026-06-30';

SELECT COUNT(*) AS matured_after_update
FROM fixed_deposit
WHERE status = 'MATURED'
    AND maturity_date <= DATE '2026-06-30';

ROLLBACK;

```

Output :

The count selected before the UPDATE should match the number of rows updated. Paste the actual count from your database.

Q25.

SQL:

```

UPDATE recurring_deposit
SET status = 'DEFAULTED'
WHERE rd_id = 8002;

UPDATE recurring_deposit
SET status = 'CLOSED'
WHERE rd_id = 8002
    AND status = 'DEFAULTED';

SELECT rd_id, status, installments_paid
FROM recurring_deposit
WHERE rd_id = 8002;

```

Output :

installments_paid remains unchanged. Closing the record rather than deleting it preserves the deposit's historical, audit and reporting record.

Level 3

Q26.

I would insist that `principal_amount` be stored as a proper numeric monetary value in the authoritative database, not `VARCHAR(30)`. The vendor should provide a canonical representation, while the ingestion/staging layer should validate and convert formats such as “2,50,000.00” and “250000 INR” before loading the clean value into the relational table. Unparseable or ambiguous records should be quarantined for correction instead of silently stored as text. This gives reliable calculations, constraints, indexing and reporting.

Q27. (Core)

I would choose `RESTRICT` rather than `ON DELETE CASCADE` for `fixed_deposit.customer_id`. When a customer leaves the bank, financial records normally need to remain available for audit, regulatory retention, reconciliation and customer service. `CASCADE` could silently delete deposit records when a customer master row is removed. `RESTRICT` forces an explicit closure or archival process instead of automatic destruction. The customer can be marked closed/inactive while the deposit history remains preserved.

Q28.

Add `authorising_employee_id` to `account_txn`, using the same integer/bigint type as `employee.employee_id`. Make it nullable because authorization is required only where applicable and old transactions may not have the information. Add a foreign key to `employee.employee_id`. For the 40 million existing rows, add the nullable column first, backfill only when reliable historical mappings exist, validate the data, and then add/validate the foreign key; do not invent authorizer IDs.

Q29. (Core)

A `CHECK` constraint is simple, fast and appropriate when the four status values are small and stable; it avoids an extra table and join. A lookup table provides a central place for status metadata such as descriptions and can be changed without editing the table constraint. A lookup table is also useful when many tables share the same status vocabulary or the list changes frequently. For this `fixed_deposit` table, I would retain the `CHECK` because only four stable statuses are described and no status metadata is required. The simpler design therefore matches the stated requirement.

Q30.

1. Stop further writes to the affected production path if this can be done safely.
2. Determine whether the `UPDATE` is still in a transaction; with autocommit on, assume it committed unless proven otherwise.
3. Identify exactly which rows changed and preserve logs/audit evidence.
4. Locate an authoritative recovery source such as audit history, backup/PITR or WAL-based recovery information.
5. Prepare a targeted corrective `UPDATE` rather than another blanket `UPDATE`.
6. Test the correction on a copy/staging database and obtain the required incident/change approval.
7. Execute the smallest safe correction and verify it independently with queries and reconciliation.
8. Record the incident, impact, recovery and verification evidence.
9. Introduce a mandatory preview/transaction workflow for bulk `UPDATE` and `DELETE` statements.
10. Require peer review or production approval for bulk data changes and use a checklist that verifies the `WHERE` clause. SQL:

Do not blame the individual; focus on recovery, controls and prevention..